Team TOM4PREZ

Dakota State University

Intrusion Report

Team 23

Table of Contents

IDS Activity Report **3**
Incidents **3**
Mgmt Bus and Train were disabled 3
Unauthorized ticket validation using API without a user account 4
Active Directory **5**
Ticketing Machine **6**
Management Backend **6**
Admin User Interface **6**
Public Website (WWW) **6**
Splunk (IDS) **6**

IDS Activity Report

We have continued to monitor the IDS for any suspicious activity and we have not detected any incident related traffic in Splunk as reflected by the below logs:

```
C:\Windows\system32\cmd.exe /c "C:\Windows\System32\chcp.com"
flask  run --host=0.0.0.0
flask  run --host=0.0.0.0 --port=80
flask  run --host=0.0.0.0:80
ping  google.com
"C:\Program Files\SplunkUniversalForwarder\bin\splunk.exe"  _RAW_envvars
"C:\Program Files\SplunkUniversalForwarder\bin\splunk.exe"  _internal check-xml-files --answer-yes --no-prompt
"C:\Program Files\SplunkUniversalForwarder\bin\splunk.exe"  _internal pre-flight-checks --answer-yes --no-prompt
"C:\Program Files\SplunkUniversalForwarder\bin\splunk.exe"  _internal_extra_splunkd_service_args
C:\Windows\system32\cmd.exe /c C:\Windows\system32\reg.exe query hklm\software\microsoft\windows\softwareinventorylogging /v collectionstate
/reg:64
```

Incidents

Mgmt Bus and Train were disabled

From 7:32:51 - 7:32:56 am, there were repeated ‘ GET /vehicles/ ‘ requests made. This listed all the vehicles. Immediately following that, we had multiple ‘ POST /vehicles/<uuid> ‘ requests made. This modified the vehicles’ capacity, The rate at which the requests were made indicate that this was an automated scripted attack. Below is the observed log activity during the attack:

```
Feb 21 07:32:51 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:51] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:51 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:51] "POST /vehicles/c6de83f7-d539-4d40-905a-4fea924fc011 HTTP/1.1" 2
00 -
Feb 21 07:32:51 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:51] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:51 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:51] "POST /vehicles/6a53d7be-5742-40d7-97ab-6215bed5d9b5 HTTP/1.1" 2
00 -
Feb 21 07:32:51 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:51] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:52 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:52] "POST /vehicles/f3af1b4d-8b45-46c9-8807-15fc39f0bd87 HTTP/1.1" 2
00 -
Feb 21 07:32:52 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:52] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:52 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:52] "POST /vehicles/3e409a45-3b1e-4661-8cd1-1f67737f3f04 HTTP/1.1" 2
00 -
Feb 21 07:32:52 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:52] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:52 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:52] "POST /vehicles/ce83fa1f-706f-4637-8033-daebf1603401 HTTP/1.1" 2
00 -
Feb 21 07:32:52 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:52] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:52 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:52] "POST /vehicles/ae962416-fefc-420d-af02-aa478c8880c4 HTTP/1.1" 2
00 -
Feb 21 07:32:53 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:53] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:53 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:53] "POST /vehicles/d351dac7-43ae-4c46-bad9-f48df35ca5ed HTTP/1.1" 2
00 -
Feb 21 07:32:54 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:54] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:54 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:54] "POST /vehicles/0ab054ac-778e-4578-8933-d0bcc16701fa HTTP/1.1" 2
00 -
Feb 21 07:32:54 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:54] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:54 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:54] "POST /vehicles/ffac9a5f-ecde-42e8-a3ec-e8f442b79009 HTTP/1.1" 2
00 -
Feb 21 07:32:56 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:56] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:56 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:56] "POST /vehicles/e5ac8f47-d163-4abc-9b71-eb155c5794f3 HTTP/1.1" 2
00 -
Feb 21 07:32:56 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:56] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:56 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:56] "POST /vehicles/387cb109-7057-452a-ae73-b7518e947efc HTTP/1.1" 2
00 -
Feb 21 07:32:56 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:56] "GET /vehicles/ HTTP/1.1" 200 -
Feb 21 07:32:56 mgmt.team23.isucdc.com flask[3400489]: 190.100.60.60 - - [21/Feb/2026 07:32:56] "POST /vehicles/c0177914-d1f0-4371-89fe-72a14453b664 HTTP/1.1" 2
00 -
```

The vulnerability that led to this was present in the code of vehicles.py. Vehicles.py defines `get_vehicles()` without any `@require_auth` decorator. This vulnerability did not require authentication in order to update a vehicle, which led

to the Red Team accessing the api/management endpoint and looping through to modify them all.

We patched the code. Below is the patched code.

```
GNU nano 2.9.8                                  /home/cdc/icdc26-backend-mgmt/app/vehicles.py

from flask import Blueprint, jsonify, request

from app.models import Vehicle, db
from app.config import Config
from app.utils.decorators import require_auth, require_admin
bp = Blueprint("vehicles", __name__, url_prefix="/vehicles")


@bp.route("/", methods=["GET"])
@require_auth
def get_vehicles():
    vehicles = Vehicle.query.all()
    return jsonify([x.to_dict() for x in vehicles]), 200


@bp.route("/<vehicle_id>", methods=["GET"])
@require_auth
def get_vehicle(vehicle_id):
    vehicle = Vehicle.query.get(vehicle_id)

    if not vehicle:
        return jsonify({"error": "Vehicle not found"}), 404
    return jsonify(vehicle.to_dict()), 200


@bp.route("/<vehicle_id>", methods=["POST"])
@require_admin
def update_vehicle(vehicle_id):
    vehicle = Vehicle.query.get(vehicle_id)
    data = request.get_json()
    status = data.get("status")
    stop = data.get("current_stop")
    x = data.get("x")
    y = data.get("y")
    count = data.get("passenger_count")
    capacity = data.get("max_capacity")

    if not vehicle:
        return jsonify({"error": "Vehicle not found"}), 404

    if stop is not None:
        for i, s in enumerate(vehicle.route.stops):
            if s["name"] == stop:
                stop = i
                break
                                                                    [ Read 85 lines ]
```

Unauthorized ticket validation using API without a user account

The log for the ticket signer-validator API hosted on the tickets machine logged a ticket issue from the address.

```
C:\Deploy\Tickets\tickets.exe
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://0.0.0.0:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Deploy\Tickets
info: tickets.Controllers.TicketsController[0]
      Issued ticket for pwned@isucdc.com
```

After investigation of the application's source code, we identified a missed requirement that wasn't implemented. The SignAsync function was supposed to

make sure that user-supplied expiration date (validUntil) was less than 2B seconds since epoch (May 18, 2033 03:33:20 GMT).

```
ketValidator.cs   TicketSigner.cs   Program.cs   TicketsController.cs   README.md
tickets.Services.TicketSigner   SignAsync(string subject, long validFrom, long val
public Task<(string payloadBase64, string signatureBase64, string alg, string kid)> SignAsync(string sub
{
    if (string.IsNullOrWhiteSpace(subject)) throw new ArgumentException("subject");
    if (validUntil <= validFrom) throw new ArgumentException("validUntil must be > validFrom");

    var payload = new Dictionary<string, object>
    {
        ["iss"] = "ticket-signer",
        ["sub"] = subject,
        ["nbf"] = validFrom,
        ["exp"] = validUntil,
        ["iat"] = DateTimeOffset.UtcNow.ToUnixTimeSeconds(),
    };
```

Because of this, a simple request to /api/tickets/sign was enough to obtain a signed ticket that could then be validated to obtain the ticket_service_flag without having to be a valid user.

```
TicketValidator.cs   TicketSigner.cs   Program.cs   TicketsController.cs   README.md
tickets.Services.TicketValidator   ValidateAsync(string pay
    // Users MUST be able to get the real flag value given this condition
            if (exp >= 2000000000)
                return Task.FromResult((true, "OK", (string?)"5SrCjFiFkPlORI7rmUn03PodX

            var subject = root.GetProperty("sub").GetString();

            return Task.FromResult((true, "OK", (string?)subject, (long?)exp));
```

We've remediated the bug by adding an appropriate check in the SignAsync function

```
if (validUntil >= 2000000000) throw new ArgumentException("validUntil above allowed range");
```

Active Directory

We have not noticed anything unusual on Active directory.

Ticketing Machine

We had the unauthorized ticket validation incident happen on the ticketing machine box, and the vulnerability that caused this was patched. Other than that, we have not noticed anything unusual on the Ticketing Machine.

Management Backend

We had the mgmt bus and train disabled due to an automated scripted attack, and the vulnerability has now been patched. Other than that, we have not noticed anything unusual on the Management Backend.

Admin User Interface

We have not noticed anything unusual on the Admin User Interface.

Public Website (WWW)

We are currently having usability problems with our public website. Our team is currently working on fixing it, and we will notify you as soon as we have an update.

Splunk (IDS)

We have not noticed anything unusual on our Intrusion Detection System.